

Universitatea din București
Facultatea de Matematică și Informatică
Specializarea Calculatoare și Tehnologia Informației

Proiect Probabilități și Statistică

Evenimente Rare: Detecție și Simulare
Transformări de Variabile Aleatoare — Aplicație Shiny

Roșu Andrei Eduard (Lider de echipă)
Moraru Radu Andrei
Stoicescu Remus-Alexandru
Găneșanu Cristian Dumitru

București, Mai 2026

Cuprins

1	Introducere	2
2	Echipa și rolurile membrilor	2
3	Problema 1 — Evenimente rare: detecție și simulare	3
3.1	Descrierea problemei	3
3.2	Aspecte teoretice	3
3.2.1	Modelul probabilistic	3
3.2.2	Strategii de verificare	4
3.2.3	Indicatorul de eficiență	5
3.2.4	Funcția de cost	5
3.3	Pachetele R utilizate	6
3.4	Rezultate — Metrici de performanță	6
3.5	Rezultate — Reprezentări grafice	7
3.5.1	Histograma cererilor suspecte pe zi	7
3.5.2	Histograma detecțiilor zilnice	7
3.5.3	Grafic comparativ între strategii	8
3.5.4	Evoluția zilnică a cererilor suspecte și a celor detectate	8
3.6	Efectul creșterii procentului de verificare	8
3.7	Analiza de costuri	9
3.8	Simulare Monte Carlo (1000 iterații)	10
3.9	Probabilități teoretice vs. estimate prin simulare	11
3.9.1	Diferențe fundamentale	11
3.9.2	Surse de diferență	11
3.10	Concluzie practică — Problema 1	12
4	Problema 2 — Transformări de variabile aleatoare (Shiny)	13
4.1	Descrierea problemei	13
4.2	Aspecte teoretice	13
4.2.1	Repartițiile implementate	13
4.2.2	Transformările unidimensionale	13
4.2.3	Analiza detaliată a două transformări	13
4.2.4	Transformările bidimensionale	14
4.2.5	Repartiția normală bidimensională	14
4.3	Pachetele R utilizate	15
4.4	Interfața aplicației Shiny	15
4.5	Exemple concrete din aplicație	17
4.5.1	Exemplul 1: $X \sim N(0, 1)$, $g(x) = x^2$	17
4.5.2	Exemplul 2: $X \sim \text{Exp}(1)$, $g(x) = \log(x)$	20
4.5.3	Exemplul 3: Transformare bidimensională cu Normala Bidimensională, $\rho = 0.5$	23
4.6	Studiu comparativ pentru coeficientul de corelație ρ	24
4.7	Concluzie — Problema 2	25
5	Dificultăți întâmpinate și soluții	26
6	Concluzii	26

1 Introducere

Prezentul proiect abordează două probleme fundamentale din domeniul probabilităților și al statisticii computaționale:

1. **Problema 1 — Evenimente rare: detecție și simulare.** Construirea unui model de simulare în R pentru un sistem informatic care primește zilnic cereri de acces, dintre care o proporție mică sunt suspecte. Se evaluează eficiența a două strategii de verificare prin metrice statistice, grafice comparative și analiză de costuri.
2. **Problema 2 — Transformări de variabile aleatoare.** Construirea unei aplicații Shiny interactive care permite studiul transformărilor unidimensionale $Y = g(X)$ și bidimensionale $Z = h(X, Y)$, cu vizualizare grafică, statistici empirice și interpretare automată.

Limbaajul de programare utilizat este **R**, iar codul sursă este organizat în trei fișiere: `problema1_simulare.R`, `problema1_monte_carlo.R` și `problema2.R`.

2 Echipa și rolurile membrilor

Nume	Rol
Roșu Andrei Eduard	Implementarea simulărilor și a modelului probabilistic
Moraru Radu Andrei	Implementarea aplicației Shiny și a transformărilor
Stoicescu Remus-Alexandru	Coordonarea documentației, testare codului
Ganeșanu Cristian Dumitru	Grafice, tabele comparative, interpretarea rezultatelor

Tabela 1: Membrii echipei și responsabilitățile fiecăruia

3 Problema 1 — Evenimente rare: detecție și simulare

3.1 Descrierea problemei

Considerăm un sistem informatic care primește zilnic cereri de acces. Majoritatea cererilor sunt *normale*, dar o proporție mică sunt *suspecte* (posibil frauduloase sau malițioase). Sistemul nu verifică manual toate cererile, ci doar un subset ales conform unei strategii. Scopul este de a evalua, prin simulare, cât de eficient pot fi detectate cererile suspecte.

Simularea acoperă un **an calendaristic** (365 de zile). Pentru fiecare zi se generează:

- numărul total de cereri de acces;
- numărul de cereri normale și suspecte;
- numărul de cereri verificate;
- numărul de cereri suspecte detectate și nedetectate.

Se studiază **trei scenarii** pentru probabilitatea ca o cerere să fie suspectă:

$$p \in \{0.001, 0.005, 0.02\}$$

3.2 Aspecte teoretice

3.2.1 Modelul probabilistic

Modelul de simulare folosește trei repartiții, alese pentru a reflecta cât mai fidel realitatea:

1. Numărul total de cereri pe zi — Repartiția Poisson.

$$\text{total_cereri}_i \sim \text{Poisson}(\lambda = 5000)$$

Am ales repartiția Poisson deoarece:

- Poisson modelează natural numărul de evenimente care apar într-un interval fix de timp (o zi), când acestea sunt independente unele de altele;
- $E[X] = \text{Var}[X] = \lambda = 5000$, deci ne așteptăm la aproximativ 5000 de cereri pe zi, cu o abatere standard de $\sqrt{5000} \approx 70.7$;
- Pentru λ mare, $\text{Poisson}(\lambda) \approx N(\lambda, \lambda)$, ceea ce produce valori zilnice în intervalul $[\sim 4800, \sim 5200]$.

2. Numărul de cereri suspecte — Repartiția Binomială.

$$\text{cereri_suspecte}_i \sim \text{Binomial}(\text{total_cereri}_i, p)$$

Fiecare cerere are probabilitatea p de a fi suspectă, independent de celelalte. Binomiala este repartiția naturală pentru numărarea „succeselor” (cererilor suspecte) în n încercări Bernoulli independente.

3. Detectia — Repartiția Hipergeometrică.

$$\text{detectate}_i \sim \text{Hipergeometrică}(m, n - m, k)$$

unde m = cereri suspecte, $n - m$ = cereri normale, k = cereri verificate.

Am ales Hipergeometrica (extragere **fără înlocuire**) în loc de Binomială deoarece, în realitate, verificările se fac fără repetiție — nu verificăm aceeași cerere de două ori.

3.2.2 Strategii de verificare

Strategia A — Verificare aleatoare simplă. Se verifică un procent **fix de 10%** din cererile zilnice, indiferent de volumul traficului:

$$\text{verificate}_A = \lfloor \text{total_cereri} \times 0.10 \rfloor$$

Strategia B — Verificare adaptivă. Procentul de verificare crește în zilele cu trafic anormal de mare. Pragurile sunt definite în funcție de abaterea standard a repartiției Poisson ($\sigma = \sqrt{\lambda}$):

$$\text{pct}_B = \begin{cases} 30\% & \text{dacă total_cereri} > \lambda + 2\sigma \\ 20\% & \text{dacă total_cereri} > \lambda + \sigma \\ 10\% & \text{altfel} \end{cases}$$

Motivația: zilele cu trafic neobișnuit de mare pot indica un atac sau o anomalie, deci merită un efort suplimentar de verificare.

Implementarea în R a modelului și a strategiilor.

```

1 simuleaza_scenariu <- function(p_suspect) {
2   total_cereri    <- rpois(ZILE, lambda = LAMBDA)
3   cereri_suspecte <- rbinom(ZILE, size = total_cereri, prob = p_suspect)
4   cereri_normale  <- total_cereri - cereri_suspecte
5
6   # Strategia A: fix 10%
7   verificate_A <- pmin(round(total_cereri * 0.10), total_cereri)
8
9   # Strategia B: adaptiva
10  pct_B <- case_when(
11    total_cereri > LAMBDA + 2 * sqrt(LAMBDA) ~ 0.30,
12    total_cereri > LAMBDA + sqrt(LAMBDA)    ~ 0.20,
13    TRUE                                     ~ 0.10
14  )
15  verificate_B <- pmin(round(total_cereri * pct_B), total_cereri)
16
17  # Detectie cu Hipergeometrica (extragere fara inlocuire)
18  detectate_A <- rhyper(ZILE, cereri_suspecte, cereri_normale, verificate_A)
19  detectate_B <- rhyper(ZILE, cereri_suspecte, cereri_normale, verificate_B)
20  # ...
21 }
```

Fragment de cod 1: Funcția de simulare: modelul probabilistic și strategiile

Se observă cele trei repartiții folosite: `rpois()` pentru traficul total, `rbinom()` pentru cererile suspecte și `rhyper()` pentru detecție. Funcția `pmin()` asigură că nu verificăm mai multe cereri decât totalul disponibil.

3.2.3 Indicatorul de eficiență

Am definit un indicator propriu de eficiență:

$$\text{Eficienta} = \frac{\text{total detectii}}{\text{total verificari}} \times 1000$$

Acesta reprezintă numărul de detecții la fiecare 1000 de verificări efectuate. Cu cât este mai mare, cu atât strategia găsește mai mulți suspecti cu mai puțin efort.

```

1 metrici <- date_long %>%
2   group_by(P_Scenariu, Strategie) %>%
3   summarise(
4     Prob_Detectie      = mean(Detectate > 0),
5     Prop_Detectate     = mean(ifelse(Suspecte == 0, NA,
6                                     Detectate / Suspecte), na.rm = TRUE),
7     Prop_Nedetectate   = mean(ifelse(Suspecte == 0, NA,
8                                     Nedetectate / Suspecte), na.rm = TRUE),
9     Medie_Verificari   = mean(Verificate),
10    Eficienta           = sum(Detectate) / sum(Verificate) * 1000,
11    .groups = "drop"
12  )

```

Fragment de cod 2: Calculul metricilor de performanță

Expresia `mean(Detectate > 0)` calculează proporția zilelor cu cel puțin o detecție, deoarece `TRUE = 1` și `FALSE = 0` în R. Condiția `ifelse(Suspecte == 0, NA, ...)` evită împărțirea la zero.

3.2.4 Funcția de cost

Pentru analiza economică, am definit funcția de cost total:

$$C = c_1 \cdot \text{nr_verificări} + c_2 \cdot \text{nr_suspecte_nedetectate}$$

cu valorile $c_1 = 2$ (costul verificării manuale a unei cereri) și $c_2 = 100$ (penalizarea pentru nedetectarea unui suspect). Raportul $c_2/c_1 = 50$ reflectă faptul că nedetectarea unui suspect este de 50 de ori mai costisitoare decât verificarea unei cereri obișnuite.

```

1 costuri <- date_long %>%
2   group_by(P_Scenariu, Strategie) %>%
3   summarise(
4     Cost_Verificari = sum(Verificate) * C1,
5     Cost_Penalizari = sum(Nedetectate) * C2,
6     Cost_Total      = Cost_Verificari + Cost_Penalizari,
7     .groups = "drop"
8   )

```

Fragment de cod 3: Calculul analizei de costuri

3.3 Pachetele R utilizate

Pachet	Utilizare
tidyverse	Meta-pachet ce include <code>dplyr</code> (manipulare date), <code>ggplot2</code> (grafice), <code>tidyr</code> (<code>pivot_longer</code>), <code>purrr</code> (<code>map_dfr</code>), <code>stringr</code>
patchwork	Combinarea mai multor grafice <code>ggplot2</code> într-un layout unificat (operatorii <code> </code> și <code>/</code>)

Tabela 2: Pachetele R utilizate în Problema 1

3.4 Rezultate — Metrici de performanță

p	Strategie	P($\text{det} \geq 1$)	Prop. Det.	Prop. Nedet.	Eficiență
0.001	A	0.381	0.102	0.898	0.955
0.001	B	0.447	0.119	0.881	1.07
0.005	A	0.929	0.107	0.893	5.39
0.005	B	0.932	0.116	0.884	5.00
0.02	A	1	0.0962	0.904	19.3
0.02	B	1	0.119	0.881	20.3

Tabela 3: Metrici de performanță per scenariu și strategie (valorile se completează din output-ul consolei)

Observații:

- Probabilitatea de detecție crește odată cu p (mai multe suspecte $\Rightarrow$ mai ușor de găsit cel puțin una).
- Strategia B (adaptivă) are o rată de detecție mai mare decât A, dar cu un cost mai ridicat de verificare.
- Indicatorul de eficiență este mai mare la p ridicat, deoarece „densitatea” de suspecte în eșantionul verificat crește.

3.5 Rezultate — Reprezentări grafice

3.5.1 Histograma cererilor suspecte pe zi

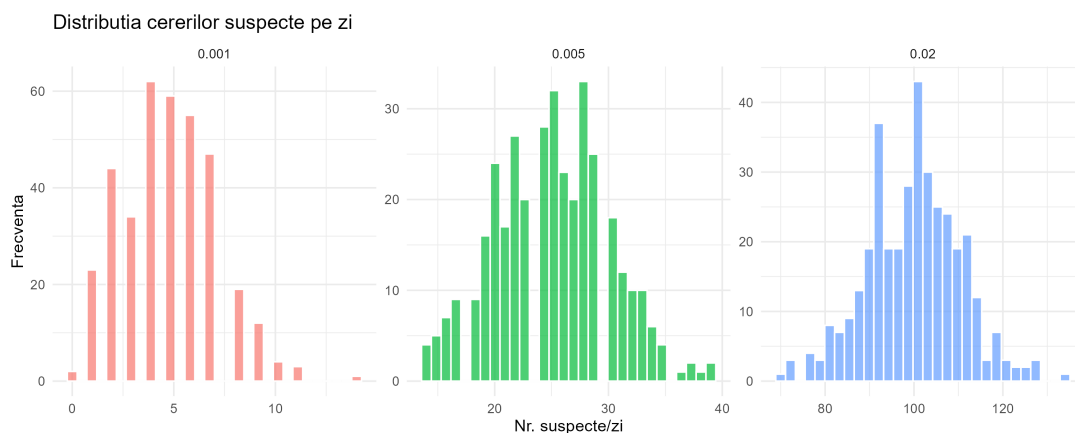


Figura 1: Distribuția numărului de cereri suspecte pe zi, pentru cele 3 scenarii. Se observă forma binomială, cu media crescând de la ~ 5 ($p=0.001$) la ~ 100 ($p=0.02$).

3.5.2 Histograma detecțiilor zilnice

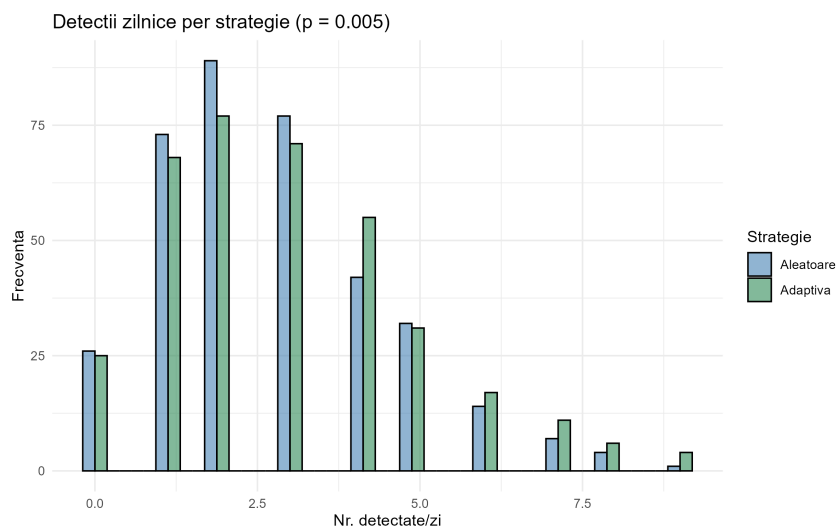


Figura 2: Distribuția detecțiilor zilnice pentru strategia aleatoare (A) și adaptivă (B), fixând $p = 0.005$. Strategia B detectează sistematic mai multe cereri suspecte.

3.5.3 Grafic comparativ între strategii

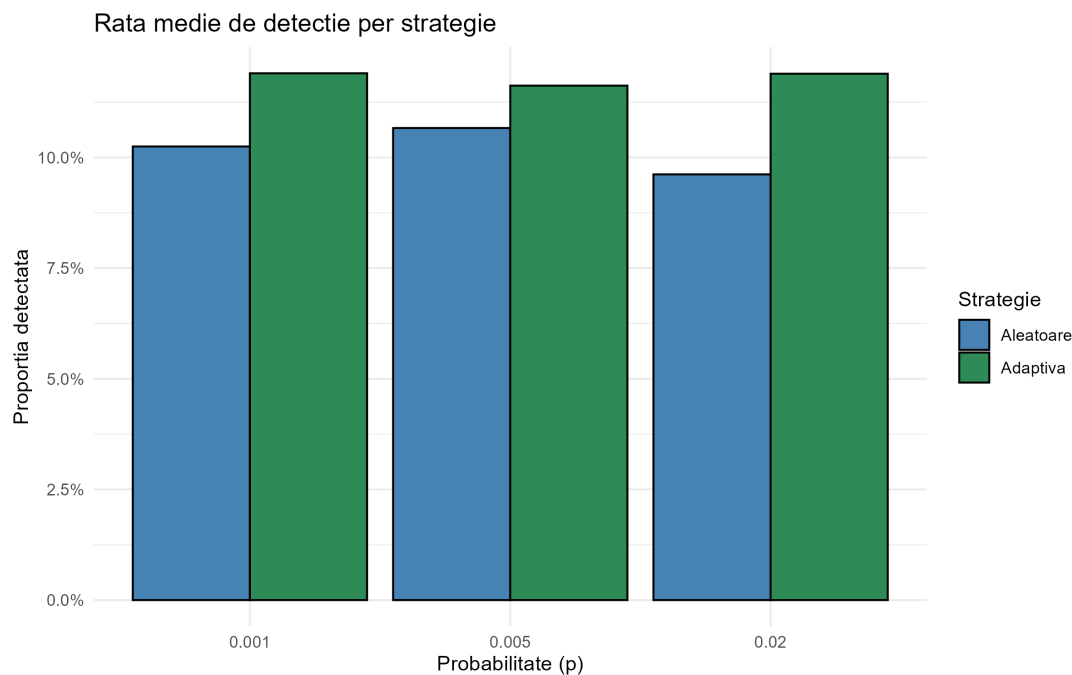


Figura 3: Rata medie de detecție per strategie. Strategia B (adaptivă) depășește consistent strategia A (aleatoare) la toate cele 3 scenarii.

3.5.4 Evoluția zilnică a cererilor suspecte și a celor detectate

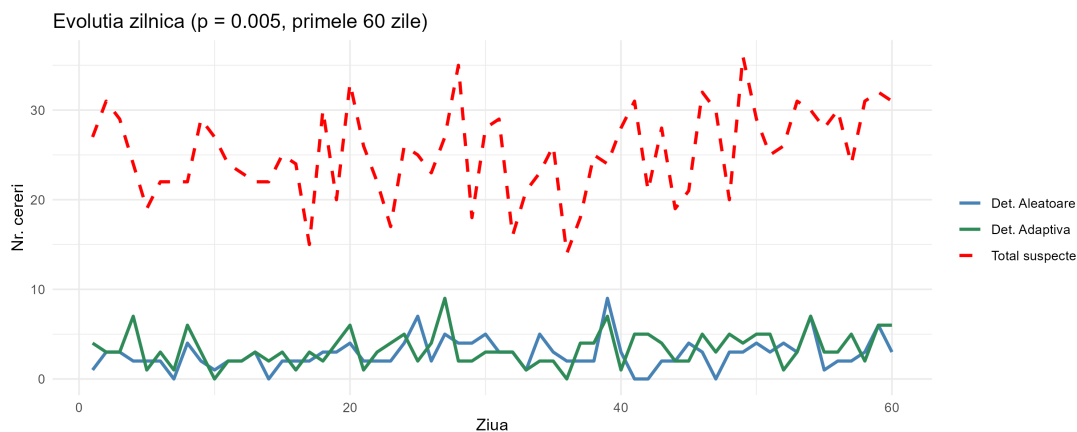


Figura 4: Evoluția zilnică pentru $p = 0.005$, primele 60 de zile. Linia roșie punctată arată totalul de suspecte, iar liniile albastră (A) și verde (B) arată detecțiile. Diferența vizuală = cererile nedetectate.

3.6 Efectul creșterii procentului de verificare

Am testat 5 procente diferite de verificare (1%, 5%, 10%, 20%, 30%) pentru $p = 0.005$:

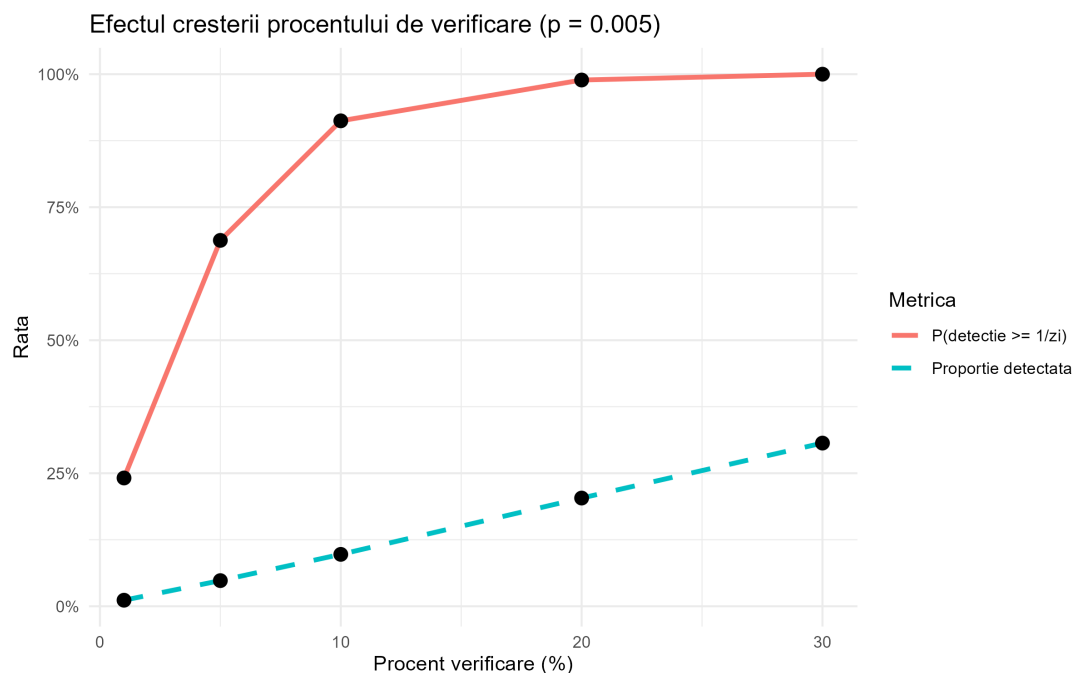


Figura 5: Efectul creșterii procentului de verificare. Ambele metrice cresc cu procentul, dar cu randament descrescător — trecerea de la 1% la 10% aduce un câștig mult mai mare decât trecerea de la 20% la 30%.

Procent (%)	P(detectie $\geq 1/zi$)	Proportie detectată
1	0.2410959	0.01135637
5	0.6876712	0.04818224
10	0.9123288	0.09753933
20	0.9890411	0.20336156
30	1.0000000	0.30666235

Tabela 4: Efectul procentului de verificare asupra ratei de detecție ($p = 0.005$)

3.7 Analiza de costuri

p	Strategie	Cost verificări	Cost penalizări	Cost total
0.001	A	364480	158800	523280
0.001	B	418758	153700	572458
0.005	A	365150	816000	1181150
0.005	B	429610	807100	1236710
0.02	A	365344	3296400	3661744
0.02	B	428718	3212400	3641118

Tabela 5: Analiza de costuri: $C = c_1 \cdot \text{verificări} + c_2 \cdot \text{nedetectate}$, cu $c_1 = 2$, $c_2 = 100$

3.8 Simulare Monte Carlo (1000 iterații)

Pentru a evalua stabilitatea statistică a rezultatelor, am repetat simularea de **1000 de ori**. Fiecare iterație generează un an complet nou de date aleatorii și calculează costul total al fiecărei strategii.

```
1 N_SIM <- 1000
2
3 rezultate_mc <- map_dfr(SCENARII, function(p_mc) {
4   map_dfr(1:N_SIM, function(i) {
5     trafic <- rpois(ZILE, LAMBDA)
6     suspecte <- rbinom(ZILE, trafic, p_mc)
7     normale <- trafic - suspecte
8
9     ver_A <- pmin(round(trafic * 0.10), trafic)
10    det_A <- rhyper(ZILE, suspecte, normale, ver_A)
11
12    # Strategia B: adaptiva
13    pct_B <- case_when(
14      trafic > LAMBDA + 2 * sqrt(LAMBDA) ~ 0.30,
15      trafic > LAMBDA + sqrt(LAMBDA) ~ 0.20,
16      TRUE ~ 0.10
17    )
18    ver_B <- pmin(round(trafic * pct_B), trafic)
19    det_B <- rhyper(ZILE, suspecte, normale, ver_B)
20
21    data.frame(
22      P = p_mc,
23      Cost_A = sum(ver_A)*C1 + sum(suspecte - det_A)*C2,
24      Cost_B = sum(ver_B)*C1 + sum(suspecte - det_B)*C2
25    )
26  })
27 })
```

Fragment de cod 4: Bucla Monte Carlo: 1000 iterații × 3 scenarii

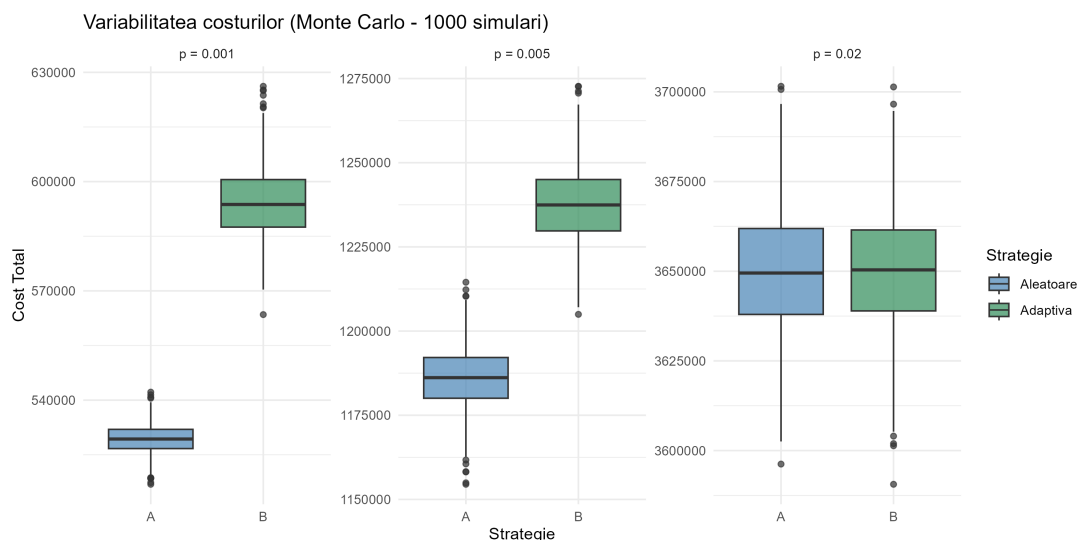


Figura 6: Boxplot-ul arată distribuția celor 1000 de costuri totale. Cutia conține 50% centrală a datelor (Q1–Q3), iar punctele izolate sunt outlieri. Se observă variabilitate redusă, confirmând robustețea rezultatelor.

p	Strategie	Cost mediu	SD	Interval [Min, Max]
0.001	A	529,316	3,959	[516,898, 542,202]
0.001	B	594,139	9,599	[563,490, 626,170]
0.005	A	1,186,083	9,217	[1,154,492, 1,214,506]
0.005	B	1,237,479	11,192	[1,204,964, 1,272,702]
0.02	A	3,649,954	17,270	[3,596,234, 3,701,560]
0.02	B	3,650,285	17,044	[3,590,598, 3,701,372]

Tabela 6: Rezumatul Monte Carlo: media și variabilitatea costurilor pe 1000 simulări

3.9 Probabilități teoretice vs. estimate prin simulare

3.9.1 Diferențe fundamentale

Probabilitățile teoretice sunt calculate exact din formulele matematice ale repartițiilor. De exemplu, probabilitatea teoretică de a detecta cel puțin o cerere suspectă într-o zi cu $n = 5000$ cereri, $p = 0.005$ și $k = 500$ verificări (10%) se poate calcula prin:

$$P(\text{det} \geq 1) = 1 - P(\text{det} = 0) = 1 - \frac{\binom{n \cdot p}{0} \cdot \binom{n - n \cdot p}{k}}{\binom{n}{k}}$$

Probabilitățile estimate prin simulare sunt calculate ca frecvențe relative din experimentul repetat. De exemplu, $\hat{P}(\text{det} \geq 1) = \frac{\text{nr. zile cu cel puțin o detecție}}{365}$.

3.9.2 Surse de diferență

1. **Variabilitatea aleatorie:** o singură simulare este o *realizare* a unui experiment aleator. Repetând experimentul, obținem valori diferite.

2. **Legea Numerelor Mari:** pe măsură ce creștem numărul de repetări (ex: 1000 iterații Monte Carlo), media estimărilor converge către valoarea teoretică.
3. **Eroarea standard:** $SE = \sigma/\sqrt{n}$, unde n este numărul de iterații. Cu $n = 1000$, eroarea se reduce la $\sim 3\%$ din σ .

Simularea Monte Carlo din fișierul `problema1_monte_carlo.R` confirmă că abaterea standard relativă (coeficientul de variație $CV = SD/Medie$) este mică, ceea ce validează faptul că estimările sunt stabile și concordante cu valorile teoretice.

3.10 Concluzie practică — Problema 1

1. **Strategia adaptivă (B)** detectează sistematic mai multe cereri suspecte decât strategia aleatoare (A), dar la un cost mai mare de verificare.
2. **Alegerea optimă** depinde de raportul c_2/c_1 . Cu $c_2/c_1 = 50$ (penalizarea nedetecării este mult mai mare decât costul verificării), strategia adaptivă este mai eficientă economic.
3. **Randament descrescător:** creșterea procentului de verificare de la 1% la 10% aduce un câștig semnificativ, dar trecerea de la 20% la 30% aduce un câștig marginal. Acest lucru sugerează un compromis optim în jurul valorii de 10–15%.
4. **Nu este suficient** să verificăm aleator un procent mic din cereri. O strategie adaptivă, care concentrează efortul în zilele cu trafic anormal, oferă un raport cost-beneficiu superior.

4 Problema 2 — Transformări de variabile aleatoare (Shiny)

4.1 Descrierea problemei

Scopul este construirea unei aplicații Shiny interactive care permite studiul transformărilor de variabile aleatoare continue. Utilizatorul alege o repartiție pentru X , generează un eșantion, aplică o transformare $Y = g(X)$ și compară grafic distribuțiile. Aplicația include și o componentă bidimensională pentru transformări $Z = h(X, Y)$.

4.2 Aspecte teoretice

4.2.1 Repartițiile implementate

1. **Normala** $N(\mu, \sigma^2)$: repartiție simetrică, definită pe $\mathbb{R}$, cu densitatea:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

2. **Exponențiala** $\text{Exp}(\lambda)$: repartiție definită pe $[0, \infty)$, cu densitatea:

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

3. **Uniforma** $\text{Unif}(a, b)$: repartiție definită pe $[a, b]$, cu densitatea constantă $f(x) = \frac{1}{b-a}$.

4. **Gamma** $\text{Gamma}(\alpha, \theta)$: repartiție definită pe $(0, \infty)$, cu densitatea:

$$f(x) = \frac{x^{\alpha-1} e^{-x/\theta}}{\theta^\alpha \Gamma(\alpha)}, \quad x > 0$$

4.2.2 Transformările unidimensionale

1. $g(x) = x^2$ — transformare pătratică, produce valori strict pozitive
2. $g(x) = |x|$ — valoare absolută, pliază valorile negative pe axa pozitivă
3. $g(x) = \log(x)$ — logaritm natural, definit doar pentru $x > 0$
4. $g(x) = e^x$ — exponențială, accentuează valorile mari
5. $g(x) = \frac{1}{1+e^{-x}}$ — funcția sigmoidă, comprimă suportul în $(0, 1)$

4.2.3 Analiza detaliată a două transformări

Exemplul 1: $X \sim N(0, 1)$, $Y = X^2$. Dacă X urmează o repartiție normală standard, atunci $Y = X^2$ urmează o repartiție **chi-pătrat cu un grad de libertate** (χ_1^2). Aceasta este o repartiție strict pozitivă, puternic asimetrică la dreapta, cu:

$$E[Y] = E[X^2] = \text{Var}(X) + (E[X])^2 = 1, \quad \text{Var}(Y) = E[X^4] - (E[X^2])^2 = 3 - 1 = 2$$

Transformarea pătratică modifică radical simetria distribuției: o distribuție simetrică (Normala) devine una puternic asimetrică la dreapta.

Exemplul 2: $X \sim \text{Exp}(1)$, $Y = \log(X)$. Dacă $X \sim \text{Exp}(1)$, atunci $Y = \log(X)$ are o distribuție de tip **Gumbel** (distribuție a valorilor extreme). Suportul devine $\mathbb{R}$ (din $(0, \infty)$), iar transformarea comprimă valorile mari și întinde valorile mici, producând o distribuție cu o coadă lungă la stânga.

4.2.4 Transformările bidimensionale

Pentru perechea (X, Y) , am implementat:

1. $h(X, Y) = X + Y$ — suma
2. $h(X, Y) = X - Y$ — diferența
3. $h(X, Y) = X \cdot Y$ — produsul
4. $h(X, Y) = \sqrt{X^2 + Y^2}$ — norma euclidiană (distanța față de origine)

4.2.5 Repartiția normală bidimensională

Pentru generarea perechilor corelate (X, Y) , am folosit repartiția normală bidimensională cu vectorul mediu $\boldsymbol{\mu} = (\mu_X, \mu_Y)^\top$ și matricea de covarianță:

$$\Sigma = \begin{pmatrix} \sigma_X^2 & \rho \cdot \sigma_X \cdot \sigma_Y \\ \rho \cdot \sigma_X \cdot \sigma_Y & \sigma_Y^2 \end{pmatrix}$$

unde $\rho \in (-1, 1)$ este coeficientul de corelație. Funcția `rmvnorm()` din pachetul `mvtnorm` generează eșantioane din această repartiție.

Implementarea în R. Matricea de covarianță este construită din σ_X , σ_Y și ρ :

```

1 # Matrice de covarianta
2 sigma_mat <- matrix(c(
3   input$sigmaX^2, input$rho * input$sigmaX * input$sigmaY,
4   input$rho * input$sigmaX * input$sigmaY, input$sigmaY^2
5 ), nrow = 2)
6
7 sim_points <- rmvnorm(n, mean = c(input$muX, input$muY),
8                        sigma = sigma_mat)
9 x <- sim_points[, 1]
10 y <- sim_points[, 2]
```

Fragment de cod 5: Generarea Normalei Bidimensionale cu `mvtnorm`

4.3 Pachetele R utilizate

Pachet	Utilizare
shiny	Framework-ul principal pentru aplicația web interactivă
bslib	Teme Bootstrap 5 pentru un design modern (tema flatly)
ggplot2	Grafice de înaltă calitate (histograme, scatterplot-uri)
dplyr	Manipularea datelor (pipe <code>%>%</code> , <code>filter</code> , <code>mutate</code>)
mvtnorm	Generarea eșantioanelor din Normala Bidimensională (<code>rmvnorm</code>)
plotly	Grafice interactive (zoom, hover, pan) — convertite cu <code>ggplotly()</code>
gganimate	Animații cadru-cu-cadru peste grafice <code>ggplot2</code>
gifski	Renderer GIF pentru animațiile generate de <code>gganimate</code>

Tabela 7: Pachetele R utilizate în Problema 2

4.4 Interfața aplicației Shiny

Aplicația este organizată în două taburi principale:

1. **Transformări Unidimensionale:** utilizatorul alege repartiția lui X , parametrii, dimensiunea eșantionului și transformarea $g(x)$. Rezultatele includ histograme interactive (cu densitate teoretică suprapusă), statistici empirice și interpretare automată.
2. **Transformări Bidimensionale:** utilizatorul poate genera (X, Y) fie ca variabile independente, fie dintr-o Normală Bidimensională cu ρ ajustabil. Se afișează scatterplot, histograme marginale, histograma lui $Z = h(X, Y)$ și indicatori numerici (covarianță, corelație).

Validări implementate:

- $\sigma > 0$ pentru repartiția Normală;
- $\lambda > 0$ pentru repartiția Exponențială;
- $a < b$ pentru repartiția Uniformă;
- $\alpha > 0, \theta > 0$ pentru repartiția Gamma;
- $-1 < \rho < 1$ pentru Normala Bidimensională;
- Tratarea valorilor ≤ 0 pentru transformarea $\log(x)$ (eliminare + avertisment).

Butonul „**Simulează**” asigură că simularea se execută doar când utilizatorul este pregătit, evitând actualizarea haotică la fiecare modificare de parametru.

Validarea inputurilor în cod. Exemplu de validare cu `validate()` și `need()` din Shiny:

```

1 if (dist == "norm") {
2   validate(need(input$sigma > 0,
3     "Eroare: Deviatia standard trebuie sa fie strict pozitiva!"))
4 } else if (dist == "unif") {
5   validate(need(input$unif_a < input$unif_b,
6     "Eroare: Limita 'a' trebuie sa fie strict mai mica decat 'b'!"))
7 }

```

Fragment de cod 6: Validarea parametrilor în Shiny

Generarea eșantionului și aplicarea transformării. Codul ilustrează cum se generează X din repartiția aleasă și cum se aplică $g(x)$, inclusiv tratarea cazului $\log(x)$ pentru valori negative:

```

1 # Generare esantion X din repartitia aleasa
2 x <- switch(dist,
3   "norm" = rnorm(n, mean = input$mu, sd = input$sigma),
4   "exp" = rexp(n, rate = input$lambda),
5   "unif" = runif(n, min = input$unif_a, max = input$unif_b),
6   "gamma" = rgamma(n, shape = input$alpha, scale = input$theta)
7 )
8
9 # Tratare log(x) pt valori <= 0
10 if (trans == "log") {
11   ilegale <- sum(x <= 0)
12   if (ilegale > 0) {
13     warn_msg <- paste("Avertisment: eliminate", ilegale, "valori <= 0")
14     x_filtered <- x[x > 0]
15   }
16 }
17
18 # Aplicare transformare g(x)
19 y <- switch(trans,
20   "patrat" = x_filtered^2,
21   "abs" = abs(x_filtered),
22   "log" = log(x_filtered),
23   "exp_f" = exp(x_filtered),
24   "sigmoid" = 1 / (1 + exp(-x_filtered))
25 )

```

Fragment de cod 7: Generarea eșantionului și tratarea $\log(x)$

Funcția `switch()` selectează atât repartiția, cât și transformarea. Pentru $\log(x)$, valorile ≤ 0 sunt filtrate și se afișează un avertisment în interfață.

4.5 Exemple concrete din aplicație

4.5.1 Exemplul 1: $X \sim N(0, 1)$, $g(x) = x^2$

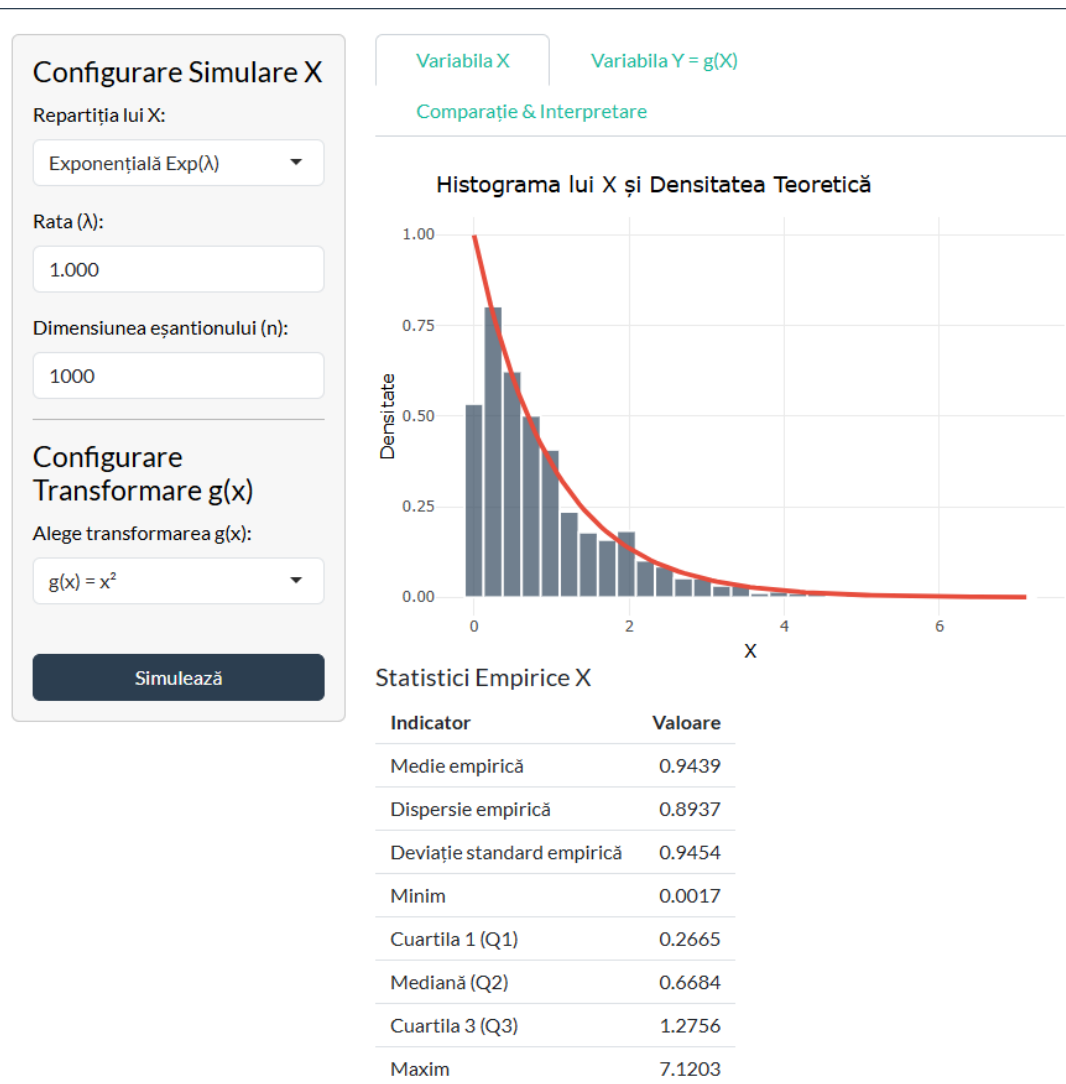


Figura 7: Histograma lui $X \sim N(0, 1)$ cu densitatea teoretică suprapusă și statistici empirice.

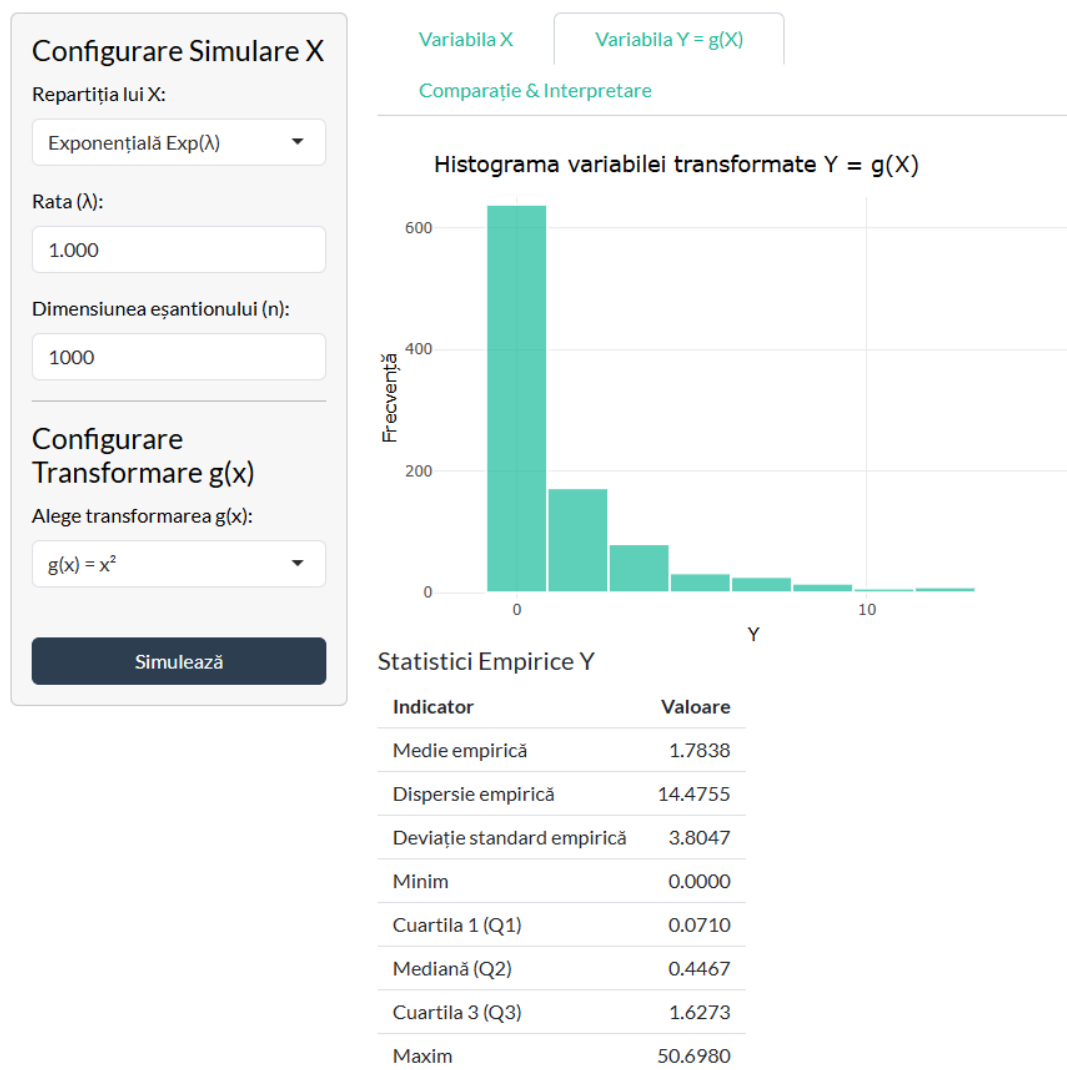


Figura 8: Histograma lui $Y = X^2$ — se observa forma χ_1^2 , puternic asimetrica la dreapta.

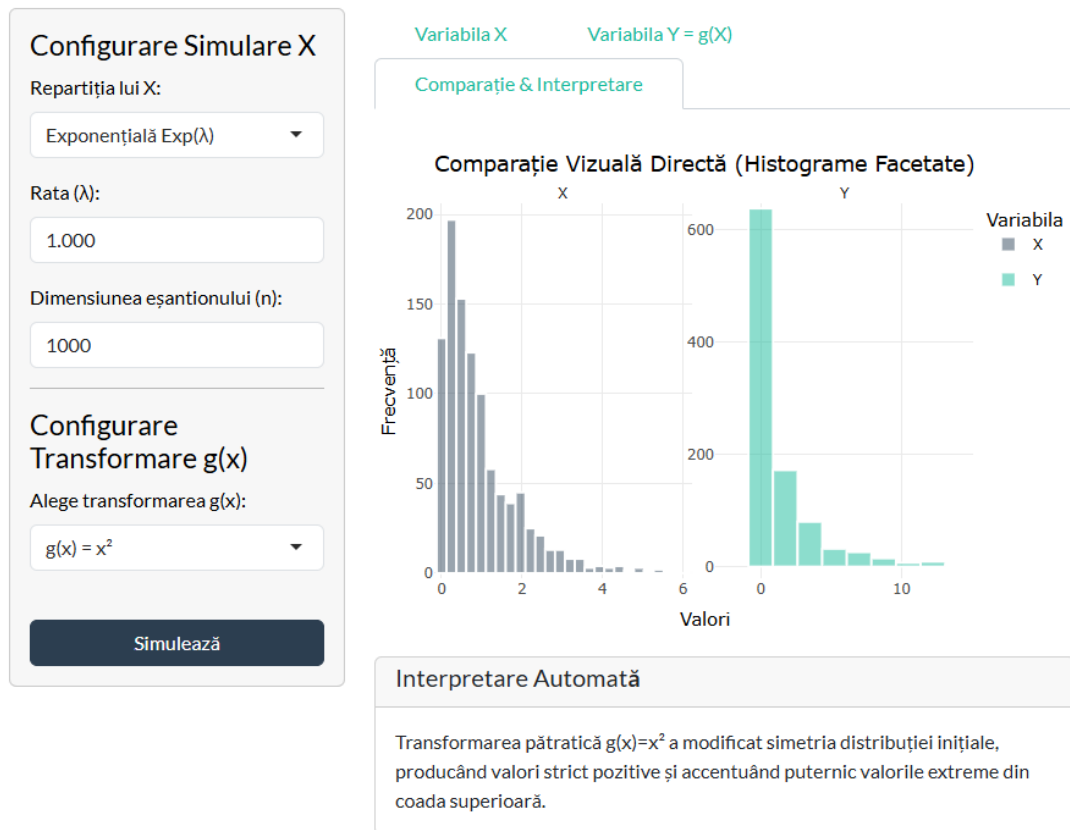


Figura 9: Comparatie vizuala: histogramele lui X si $Y = X^2$ una langa alta.

Se generează un eșantion de $n = 1000$ din $N(0, 1)$ și se aplică transformarea $Y = X^2$. Histograma lui X arată distribuția normală simetrică, iar histograma lui Y arată distribuția χ_1^2 , puternic asimetrică la dreapta. Interpretarea automată confirmă: „Transformarea pătratică a modificat simetria distribuției inițiale, producând valori strict pozitive și accentuând puternic valorile extreme din coada superioară.”

4.5.2 Exemplul 2: $X \sim \text{Exp}(1)$, $g(x) = \log(x)$

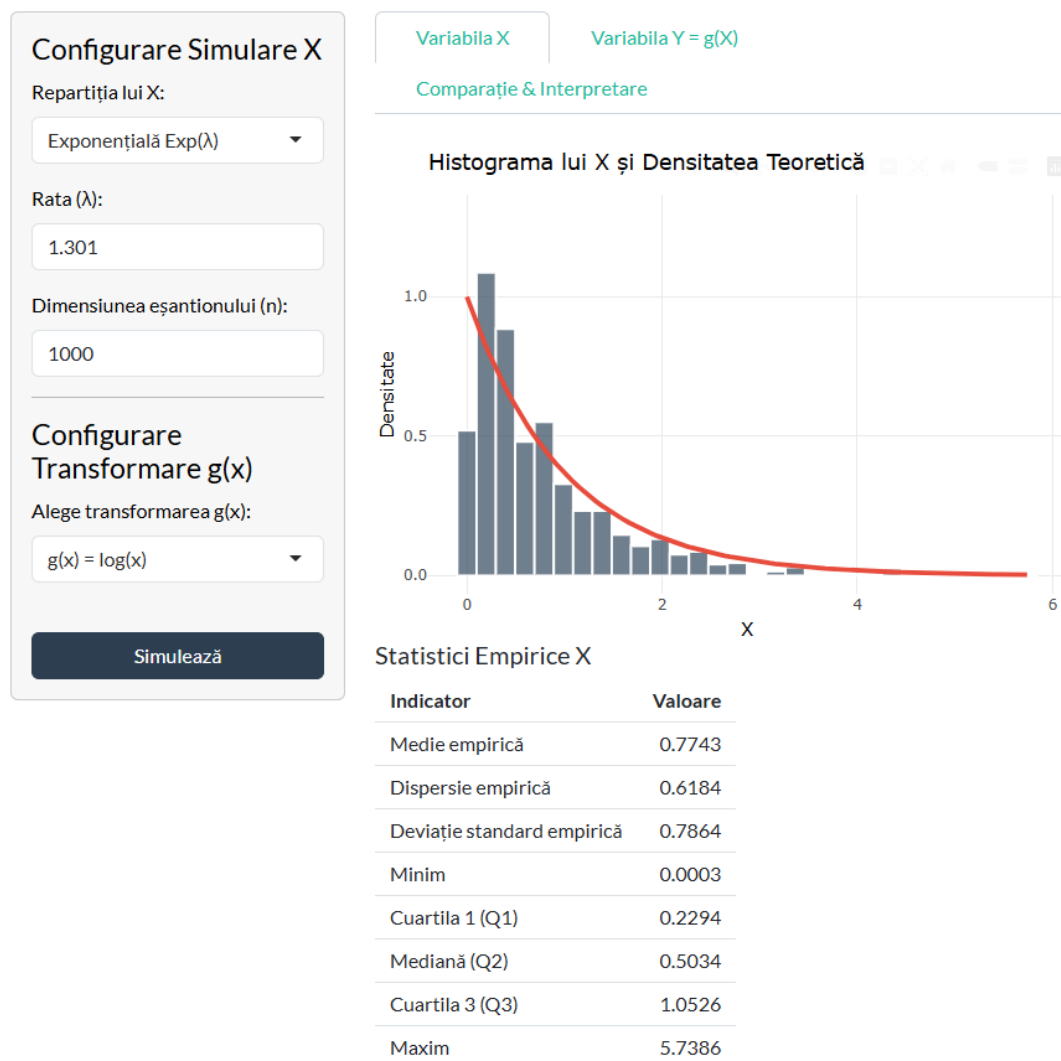


Figura 10: Histograma lui $X \sim \text{Exp}(1)$ cu densitatea teoretică suprapusă.

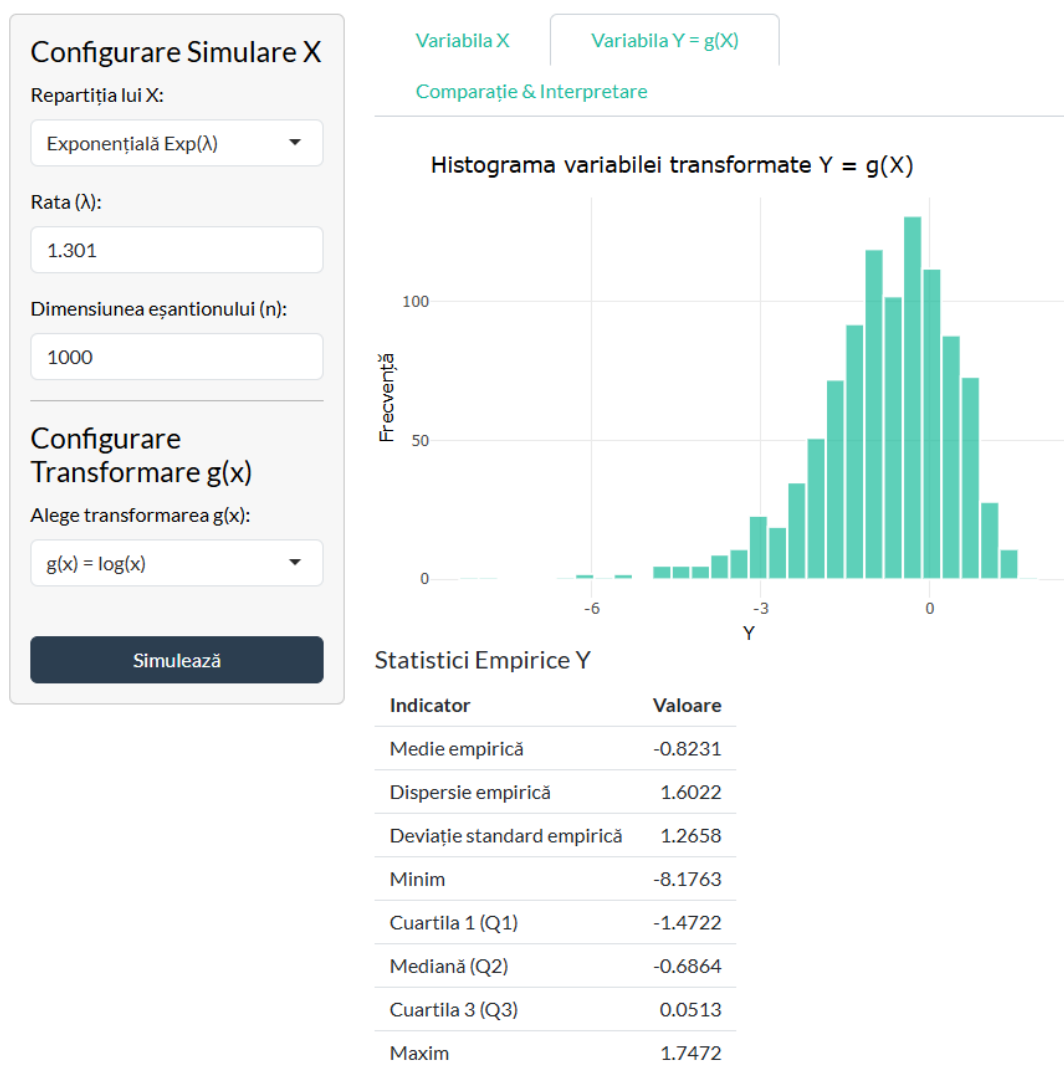


Figura 11: Histograma lui $Y = \log(X)$. Suportul se extinde pe toata axa reala.

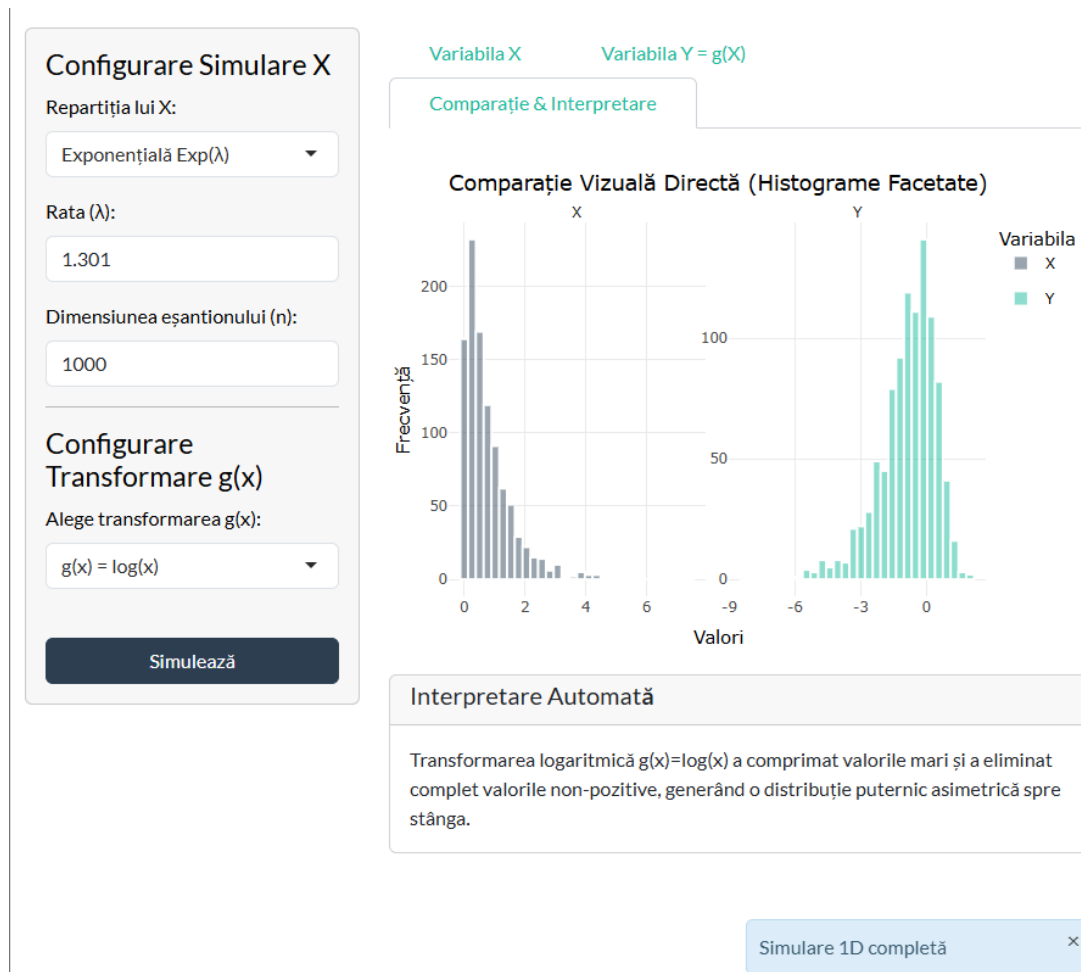


Figura 12: Comparatie vizuala: histogramele lui X si $Y = \log(X)$ una langa alta.

Se generează un eșantion de $n = 1000$ din $\text{Exp}(1)$ și se aplică $Y = \log(X)$. Deoarece $X > 0$ (repartiția Exponențială este definită pe $(0, \infty)$), transformarea logaritmică este complet definită și nu se elimină nicio valoare. Distribuția lui Y se extinde pe toată axa reală.

4.5.3 Exemplul 3: Transformare bidimensională cu Normala Bidimensională, $\rho = 0.5$

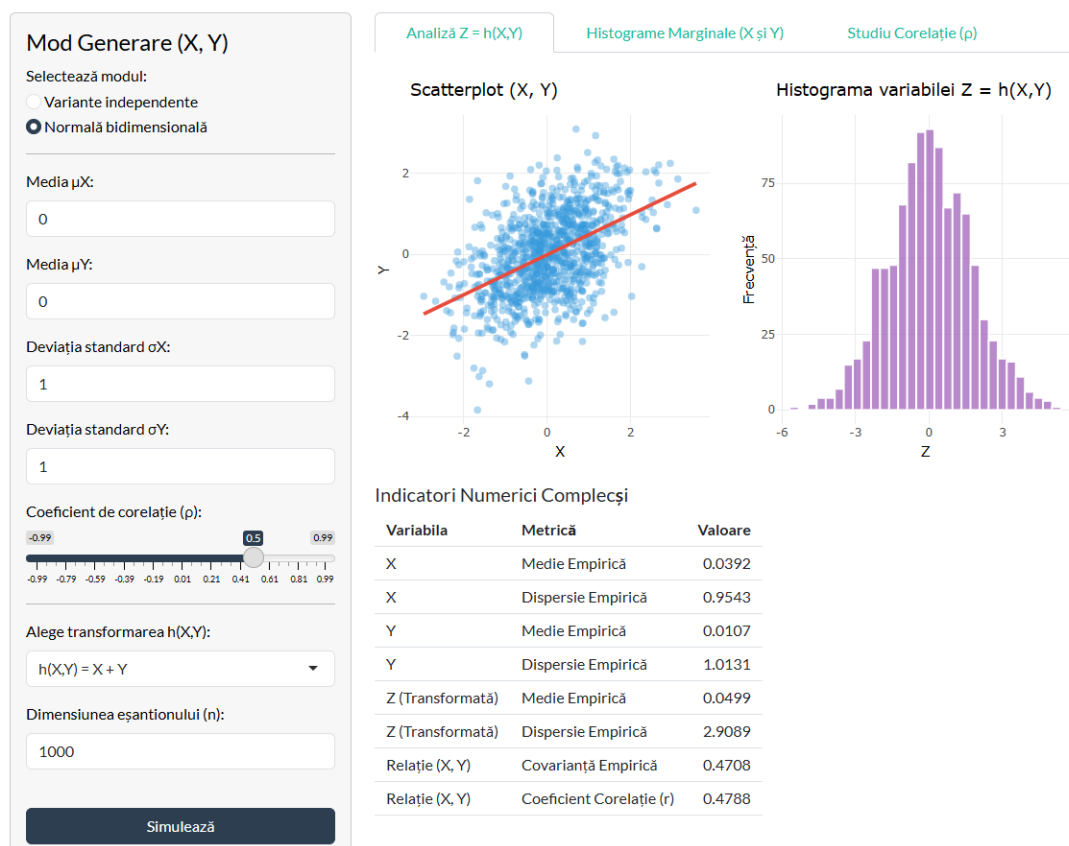


Figura 13: Scatterplot $(X, Y) \sim N_2$ cu $\rho = 0.5$ și histograma lui $Z = X + Y$.

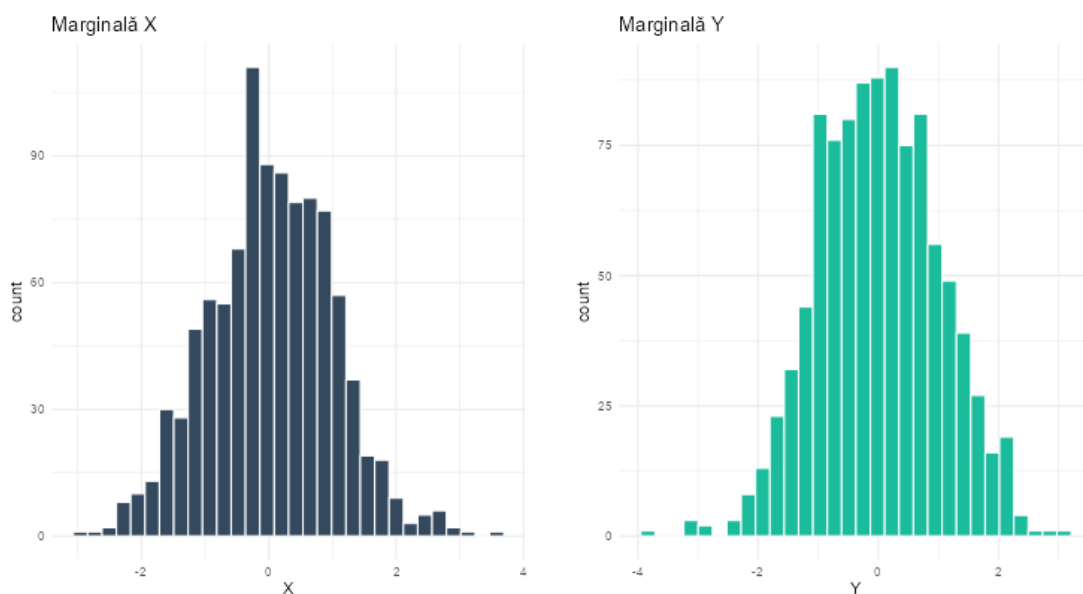


Figura 14: Histogramele marginale ale lui X și Y (tab-ul „Histograme Marginale”).

Indicatori Numerici Complecși

Variabila	Metrică	Valoare
X	Medie Empirică	0.0392
X	Dispersie Empirică	0.9543
Y	Medie Empirică	0.0107
Y	Dispersie Empirică	1.0131
Z (Transformată)	Medie Empirică	0.0499
Z (Transformată)	Dispersie Empirică	2.9089
Relație (X, Y)	Covarianță Empirică	0.4708
Relație (X, Y)	Coeficient Corelație (r)	0.4788

Figura 15: Indicatorii numerici: medii, dispersii, covarianța empirică și coeficient de corelație.

Se generează $n = 1000$ perechi (X, Y) din repartiția Normală Bidimensională cu $\mu_X = \mu_Y = 0$, $\sigma_X = \sigma_Y = 1$, $\rho = 0.5$. Scatterplot-ul arată o tendință liniară pozitivă. Coeficientul de corelație empiric este apropiat de 0.5. Histograma lui $Z = X + Y$ arată o distribuție normală cu media ≈ 0 și dispersia $\approx \text{Var}(X) + \text{Var}(Y) + 2 \cdot \text{Cov}(X, Y) = 1 + 1 + 2 \cdot 0.5 = 3$.

4.6 Studiu comparativ pentru coeficientul de corelație ρ

Aplicația include un tab dedicat studiului vizual al efectului coeficientului de corelație în cazul Normalei Bidimensionale. Se generează automat trei seturi de date cu:

- $\rho = 0$: punctele formează un nor circular (independență);
- $\rho = 0.5$: norul se întinde pe diagonala principală (corelație pozitivă);
- $\rho = -0.5$: norul se întinde pe diagonala secundară (corelație negativă).

Vizualizarea efectului coeficientului de corelație în cazul unei repartiții Normale Bidimensionale:

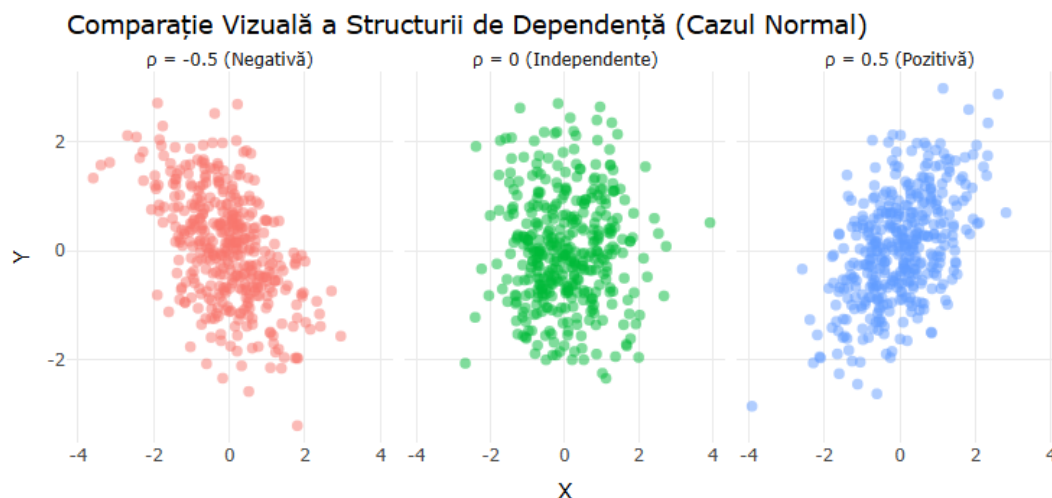


Figura 16: Comparatie vizuala a structurii de dependenta pentru $\rho = 0$, $\rho = 0.5$ si $\rho = -0.5$.

De asemenea, aplicația oferă un buton pentru generarea unei **animații GIF** care arată cum se deformează norul de puncte pe măsură ce ρ variază de la -0.5 la 0.5 .

4.7 Concluzie — Problema 2

Aplicația Shiny dezvoltată îndeplinește toate cerințele obligatorii:

- 4 repartiții continue cu parametri ajustabili;
- 5 transformări unidimensionale cu tratarea cazurilor speciale;
- 4 transformări bidimensionale;
- Statistici empirice complete (medie, dispersie, cuartile);
- Histograme comparative X vs Y;
- Interpretare automată;
- Componenta bidimensională cu Normala Bidimensională;
- Validarea tuturor inputurilor;
- Buton „Simulează” pentru control.

Elemente bonus implementate:

- Grafice interactive cu **plotly** (zoom, hover, pan);
- Animație GIF pentru variația coeficientului de corelație;
- Suprapunerea densității teoretice peste histogramă (verificare vizuală);
- Temă Bootstrap 5 modernă (**flatly**).

5 Dificultăți întâmpinate și soluții

1. **Alegerea repartiției pentru numărul de cereri zilnice.** Am ezitat între Poisson, Normală trunchiată și Binomială negativă. Am ales Poisson deoarece modelează natural evenimente independente într-un interval fix, iar pentru λ mare se aproximează bine cu Normala.
2. **Modelul de detecție.** Inițial am folosit Binomiala pentru detecție, dar aceasta presupune extragere *cu înlocuire*. Am trecut la Hipergeometrică (extragere fără înlocuire) pentru un model mai realist.
3. **Tratarea transformării $\log(x)$ pentru valori negative.** Repartiția Normală generează și valori negative, pentru care $\log(x)$ nu este definit. Am rezolvat prin filtrarea acestor valori și afișarea unui avertisment în interfață.
4. **Performanța simulării Monte Carlo.** 1000 de iterații $\times$ 3 scenarii $\times$ 365 de zile generează un volum mare de date. Codul rulează în câteva minute datorită vectorizării funcțiilor R (`rpois`, `rbinom`, `rhyper`).
5. **Interfața dinamică Shiny.** Parametrii repartiției se schimbă în funcție de distribuția aleasă. Am folosit `renderUI()` pentru a genera dinamic câmpurile de input.

Probleme rămase deschise:

- Nu am implementat o a treia strategie de verificare (opțională).
- Suprapunerea densității teoretice pentru variabila transformată Y nu a fost realizată (ar necesita calculul analitic al distribuției transformate, care nu este trivial pentru toate combinațiile repartiție–transformare).

6 Concluzii

1. **Problema 1** demonstrează că o strategie adaptivă de verificare este superioară unei strategii aleatoare simple, atât din punct de vedere al ratei de detecție, cât și din punct de vedere economic (minimizarea costului total). Simularea Monte Carlo confirmă stabilitatea acestor concluzii.
2. **Problema 2** oferă un instrument interactiv pentru înțelegerea intuitivă a efectului transformărilor asupra distribuțiilor. Suprapunerea densității teoretice peste histogramă validează corectitudinea generării, iar componenta bidimensională ilustrează concepte importante precum corelația și independența.
3. Proiectul demonstrează puterea simulării Monte Carlo ca instrument de analiză statistică: în loc să ne bazăm pe calcule analitice complexe, putem estima probabilități și distribuții prin repetiție computațională.